

AI Lab assignment 6
Tejas Patil
U24AI061

Q1)

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;

vector<string> city = {
    "Chicago", "Indianapolis", "Columbus", "Cleveland", "Detroit",
    "Buffalo", "Pittsburgh", "Baltimore", "Philadelphia",
    "New York", "Boston", "Providence", "Syracuse", "Portland"
};

int n = 14;

/* Heuristic: Straight-line distance to Boston */
vector<int> h = {
    860, // Chicago
    780, // Indianapolis
    640, // Columbus
    550, // Cleveland
    610, // Detroit
    400, // Buffalo
    470, // Pittsburgh
    360, // Baltimore
    270, // Philadelphia
    215, // New York
    0,   // Boston
    50,  // Providence
    260, // Syracuse
    107  // Portland
};

vector<vector<int>> dist(n, vector<int>(n, INF));
```

```

vector<vector<int>>> adj(n);

class prio_queue {
private:
    pair<int, vector<int>>> heap[2000];
    int sz = 0;

    bool better(pair<int, vector<int>>> a,
                pair<int, vector<int>>> b) {
        return a.first < b.first;
    }

    void siftUp(int i) {
        while (i > 0) {
            int p = (i - 1) / 2;
            if (better(heap[p], heap[i])) break;
            swap(heap[p], heap[i]);
            i = p;
        }
    }

    void siftDown(int i) {
        while (true) {
            int l = 2*i + 1, r = 2*i + 2, best = i;
            if (l < sz && better(heap[l], heap[best])) best = l;
            if (r < sz && better(heap[r], heap[best])) best = r;
            if (best == i) break;
            swap(heap[i], heap[best]);
            i = best;
        }
    }

public:
    void pushNode(int cost, vector<int> path) {
        heap[sz] = {cost, path};
        siftUp(sz);
        sz++;
    }

    pair<int, vector<int>>> topNode() {

```

```

        return heap[0];
    }

    void popNode() {
        heap[0] = heap[sz - 1];
        sz--;
        siftDown(0);
    }

    bool khaliHai() {
        return sz == 0;
    }
};

void printPath(vector<int> path) {
    int total = 0;
    for (int i = 0; i + 1 < path.size(); i++) {
        cout << city[path[i]] << " -> ";
        total += dist[path[i]][path[i+1]];
    }
    cout << city[path.back()] << "\n";
    cout << "TOTAL COST = " << total << " miles\n";
}

void GreedyBestFirst(int src, int dest) {
    cout << "\n===== GREEDY BEST FIRST SEARCH =====\n";

    prio_queue pq;
    vector<bool> visited(n, false);
    vector<int> expanded;

    pq.pushNode(h[src], {src});

    while (!pq.khaliHai()) {
        auto cur = pq.topNode();
        pq.popNode();

        vector<int> path = cur.second;
        int u = path.back();
    }
}

```

```

        if (visited[u]) continue;
        visited[u] = true;
        expanded.push_back(u);

        if (u == dest) {
            cout << "PATH:\n";
            printPath(path);

            cout << "Cities Visited (Expansion Order):\n";
            for (int x : expanded)
                cout << city[x] << " ";
            cout << "\n";
            return;
        }

        for (int v : adj[u]) {
            if (!visited[v]) {
                auto newPath = path;
                newPath.push_back(v);
                pq.pushNode(h[v], newPath);
            }
        }
    }

    cout << "No Path Found\n";
}

void AStar(int src, int dest) {
    cout << "\n==== A* SEARCH =====\n";

    prio_queue pq;
    vector<bool> visited(n, false);
    vector<int> expanded;

    pq.pushNode(h[src], {src});

    while (!pq.khaliHai()) {
        auto cur = pq.topNode();
        pq.popNode();
    }
}

```

```

        vector<int> path = cur.second;
        int u = path.back();

        if (visited[u]) continue;
        visited[u] = true;
        expanded.push_back(u);

        if (u == dest) {
            cout << "PATH:\n";
            printPath(path);

            cout << "Cities Visited (Expansion Order):\n";
            for (int x : expanded)
                cout << city[x] << " ";
            cout << "\n";
            return;
        }

        for (int v : adj[u]) {
            if (!visited[v]) {
                auto newPath = path;
                newPath.push_back(v);

                int g = 0;
                for (int i = 0; i + 1 < newPath.size(); i++)
                    g += dist[newPath[i]][newPath[i+1]];

                int f = g + h[v];
                pq.pushNode(f, newPath);
            }
        }
    }

    cout << "No Path Found\n";
}

int main() {

    for (int i = 0; i < n; i++)
        dist[i][i] = 0;

```

```

dist[0][4] = dist[4][0] = 283;
dist[0][3] = dist[3][0] = 345;
dist[0][1] = dist[1][0] = 182;
dist[1][2] = dist[2][1] = 176;
dist[2][3] = dist[3][2] = 144;
dist[2][6] = dist[6][2] = 185;
dist[3][4] = dist[4][3] = 169;
dist[3][6] = dist[6][3] = 134;
dist[3][5] = dist[5][3] = 189;
dist[4][5] = dist[5][4] = 256;
dist[5][12] = dist[12][5] = 150;
dist[5][6] = dist[6][5] = 215;
dist[6][8] = dist[8][6] = 305;
dist[6][7] = dist[7][6] = 247;
dist[7][8] = dist[8][7] = 101;
dist[8][9] = dist[9][8] = 97;
dist[8][12] = dist[12][8] = 253;
dist[9][10] = dist[10][9] = 215;
dist[9][11] = dist[11][9] = 181;
dist[10][11] = dist[11][10] = 50;
dist[10][13] = dist[13][10] = 107;
dist[12][10] = dist[10][12] = 312;

for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        if (dist[i][j] != INF && i != j)
            adj[i].push_back(j);

int src = 0;    // Chicago
int dest = 10;  // Boston

GreedyBestFirst(src, dest);
AStar(src, dest);

return 0;
}

```

===== GREEDY BEST FIRST SEARCH =====

PATH:

Chicago -> Cleveland -> Buffalo -> Syracuse -> Boston

TOTAL COST = 996 miles

Cities Visited (Expansion Order):

Chicago Cleveland Buffalo Syracuse Boston

===== A* SEARCH =====

PATH:

Chicago -> Cleveland -> Buffalo -> Syracuse -> Boston

TOTAL COST = 996 miles

Cities Visited (Expansion Order):

Chicago Detroit Cleveland Buffalo Syracuse Pittsburgh Indianapolis Boston

Q2)

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;

int n = 5;

// 0 = empty
// 1 = obstacle
// 2 = start
// 3 = reward
vector<vector<int>> grid = {
    {2,0,0,0,1},
    {0,1,0,0,3},
    {0,3,0,1,1},
    {0,1,0,0,1},
    {3,0,0,0,3}
};

class prio_queue {
private:
```

```

pair<int, vector<pair<int,int>>> heap[1000];
int sz = 0;

bool better(pair<int, vector<pair<int,int>>> a,
            pair<int, vector<pair<int,int>>> b) {
    return a.first < b.first; // smaller f(n) first
}

void siftUp(int i) {
    while (i > 0) {
        int p = (i - 1) / 2;
        if (better(heap[p], heap[i])) break;
        swap(heap[p], heap[i]);
        i = p;
    }
}

void siftDown(int i) {
    while (true) {
        int l = 2*i + 1, r = 2*i + 2, best = i;
        if (l < sz && better(heap[l], heap[best])) best = l;
        if (r < sz && better(heap[r], heap[best])) best = r;
        if (best == i) break;
        swap(heap[i], heap[best]);
        i = best;
    }
}

public:
    void pushNode(int cost, vector<pair<int,int>> path) {
        heap[sz] = {cost, path};
        siftUp(sz);
        sz++;
    }

    pair<int, vector<pair<int,int>>> topNode() {
        return heap[0];
    }

    void popNode() {

```



```

        heap[0] = heap[sz - 1];
        sz--;
        siftDown(0);
    }

    bool khaliHai() {
        return sz == 0;
    }
};

// Manhattan heuristic to nearest reward
int heuristic(int x, int y, vector<pair<int,int>> rewards) {
    int h = INF;
    for(auto r : rewards) {
        h = min(h, abs(x - r.first) + abs(y - r.second));
    }
    return h;
}

void AStar() {

    prio_queue pq;

    vector<pair<int,int>> rewards;
    pair<int,int> start;

    for(int i=0;i<n;i++) {
        for(int j=0;j<n;j++) {
            if(grid[i][j] == 2)
                start = {i,j};
            if(grid[i][j] == 3)
                rewards.push_back({i,j});
        }
    }

    vector<pair<int,int>> finalPath;

    pair<int,int> current = start;
    int totalCost = 0;

```

```

while(!rewards.empty()) {

    pq = prio_queue(); // reset pq
    vector<vector<bool>> visited(n, vector<bool>(n,false));

    pq.pushNode(0, {current});

    while(!pq.khaliHai()) {

        auto cur = pq.topNode();
        pq.popNode();

        int f = cur.first;
        vector<pair<int,int>> path = cur.second;
        auto node = path.back();

        int x = node.first;
        int y = node.second;

        if(visited[x][y]) continue;
        visited[x][y] = true;

        // check if reward
        bool found = false;
        for(int i=0;i<rewards.size();i++){
            if(rewards[i] == node){
                rewards.erase(rewards.begin()+i);
                found = true;
                break;
            }
        }

        if(found) {
            totalCost += path.size() - 1;

            for(auto p : path)
                finalPath.push_back(p);

            current = node;
            break;
        }
    }
}

```

```

    }

    int dx[4] = {-1,1,0,0};
    int dy[4] = {0,0,-1,1};

    for(int k=0;k<4;k++) {
        int nx = x + dx[k];
        int ny = y + dy[k];

        if(nx>=0 && ny>=0 && nx<n && ny<n &&
            grid[nx][ny] != 1 &&
            !visited[nx][ny]) {

            auto newPath = path;
            newPath.push_back({nx,ny});

            int g = newPath.size() - 1;
            int h = heuristic(nx, ny, rewards);
            int fn = g + h;

            pq.pushNode(fn, newPath);
        }
    }
}

cout << "A* Path Visiting All Rewards:\n";
for(auto p : finalPath)
    cout << "(" << p.first << "," << p.second << ") -> ";

cout << "END\n";
cout << "TOTAL COST = " << totalCost << endl;
}

int main() {
    AStar();
    return 0;
}

```

```
AI Lab\Assignment 6\tempCodeRunnerFile
```

```
A* Path Visiting All Rewards:
```

```
(0,0) -> (1,0) -> (2,0) -> (2,1) -> (2,1) -> (2,0) -> (3,0) -> (4,0) -> (4,0) -> (4,1) -> (4,2) ->  
(4,3) -> (4,4) -> (4,4) -> (4,3) -> (3,3) -> (3,2) -> (2,2) -> (1,2) -> (1,3) -> (1,4) -> END
```

```
TOTAL COST = 17
```